

Team Coral AI Usage Report

AI Tool Used: Gemini 3.1 Pro

1. Prompting Strategies

Our team utilized a highly specific, context-driven, and multimodal prompting strategy to interact with the generative AI tool. Rather than asking open-ended programming questions, we framed our prompts around explicit goals and provided relevant contextual constraints. For example, when addressing data cleaning, we went through the dataset and discovered that missing values were represented by dashes rather than standard NA markers. Consequently, as seen in Interaction 3, we specified the characteristics of the dataset (Changi station historical daily weather CSV) and instructed the AI to identify missing data by targeting troublesome characters, such as em-dashes (–) and en-dashes (‑).

We also actively designated the AI as a critical reviewer. Referring to Interaction 2, when drafting our design brief, we presented our initial target audience and goals, explicitly asking, “Is this audience and task clear enough? suggest how we can make it more specific.” This forced the model to evaluate our logic rather than just generate text. Additionally, we incorporated visual aids and real-world feedback into our prompts as observed in Interaction 1 and Interaction 10. When an initial visualisation proved suboptimal, we shared that context directly with the AI. For instance, in Interaction 10, after our professor noted that our generated stacked bar chart was “visually cluttering,” we uploaded a screenshot of the stacked bar chart alongside the professor’s exact critique. Providing both visual evidence and context allowed the AI to understand the design flaw and suggest more effective and purposeful alternatives.

Finally, we guided the AI by dictating the specific R packages we intended to use, such as requesting solutions utilising `dplyr`, `tidyr`, and `lubridate`. By defining the expected output formats, we reduced AI hallucinations and minimised the need for multiple iterative corrections.

2. Influence on Design Decisions

The AI tool significantly shaped our data processing pipeline and conceptual framework in three specific instances:

- **Refining Target Audience and Task Scope:** Initially, our design brief targeted a broad audience of students and residents for general historical observation as seen in Interaction 2. By asking the AI to critique our task clarity, the model highlighted a critical difference between casual observation and decision-focused information, introducing urban planners and policymakers into the discussion. This insight fundamentally shifted our project direction. We ultimately pivoted to make urban planners, policymakers, and environmental agencies our primary audience. Consequently, we redesigned our visualisation’s core objective to highlight

heavy and extreme rainfall periods across months and years, specifically to support practical applications like drainage planning, flood-prevention discussions, and public communication.

- **Handling Non-Standard NA Characters:** We originally planned to handle missing values by loading the raw data and running custom data transformation scripts across all columns. However, the AI suggested a change in the pipeline for data transformation: filtering specific Unicode characters directly via the `na.strings` parameter within the `read.csv` function in Interaction 3. We felt that this is an improvement because it filtered known artifacts at the data loading stage, simplifying transformations later in the pipeline.
- **Addressing Implicit Missing Data:** We needed a way to identify missing months in the time series and originally considered complicated grouping logic. The AI recommended generating a definitive grid of all possible year-month combinations using `tidyr::expand_grid`, and then left-joining our observed data against this master grid as shown in Interaction 5. We adopted this relational-join strategy because it provided a mathematically rigorous way to expose implicitly missing rows as `NA` rows, which was critical for our subsequent visualisation steps.

3. Corrections and Rejections

While the AI provided strong conceptual architectures, we had to critically refine its implementation to ensure it met our goals and coding standards.

- **Rejecting Ineffective Visualisations:** In Interaction 10, after we provided feedback that our stacked bar chart was visually cluttering, the AI suggested alternative ideas: faceted line charts, area under the graph, and lollipop charts. We coded these suggestions but ultimately rejected all of them. We realised they were not intuitive and failed to solve our primary target: identifying exactly when heavy and extreme rain periods occur across months and years. Rather than asking the AI for more blind generations, we conducted independent research and discovered heatmaps. We returned to the AI to correct its direction, pitching the heatmap as a fresh take alongside our specific ideas for the `x` and `y` axes as seen in Interaction 11 and 12. The AI validated this approach, and we shifted to using the tool to execute our independently researched design rather than relying on it for ideation.
- **Rejecting Scope Creep and Monolithic Helper Functions:** In Interaction 3, the AI provided a script to clean non-numeric text by defining a standalone `parse_base_number` helper function, and suggested applying it across nine different columns, including wind speed and temperature using `dplyr::across`. We rejected this implementation because it cluttered the script with variables completely outside our project's scope and created an unnecessary external function that made debugging harder when it failed. Instead of adopting the AI's complicated architecture, we handled it by writing a simple, inline regular expression (`gsub`) directly within our `mutate()` pipeline, and restricted it to apply on the `daily_rainfall_total_mm` column. This ensured our code remained clean, efficient, and strictly aligned with our project's goals.

4. Ethical and Accountable Use

We maintained accountability by independently verifying all factual and technical claims generated by the AI. We reviewed the AI's suggested data transformation pipelines in Interaction 3, 4 and 5 to ensure they did not blindly hide underlying data errors. We integrated the AI's contextual

warning that our dataset only represents Changi station, clearly noting in our project interpretation that the results should not be mistaken for a national average to prevent geographical bias. As for attribution, the AI was utilised as an advisor to critique and refine the phrasing of our design brief (Interaction 2) and to brainstorm initial R architectures, but all final report text and project code were developed and heavily modified by the team. Finally, the main limitation we observed was the AI's tendency to produce "reasonable but ineffective" code. While the model successfully generated error-free scripts for stacked bar charts (Interaction 9) or standalone helper functions (Interaction 3, 4 and 5), these outputs often lacked human intuition, resulting in visually cluttered designs or overly complicated software architectures that required strict human intervention to correct.

5. Tool Specification

A single generative AI conversational model (**Gemini 3.1 Pro**) was utilised across all interactions documented in `ai-log.md` for R script generation, logic troubleshooting, and visualisation improvements.

Total word count of AI Usage Report: 1073 words.